REPORT · 2026年10月18日

扫描件场景缺陷排查与修复

H3D v1.1.0 · r6.3 场景① 实机复盘

文保数字化研发组
2026年10月18日

目 录

一、排查结论 4

1.1 七项缺陷一览 4

二、排查方法：把场景② 的问题当探针 5

三、D1：只有扫描件时整尊失败（致命） 6

3.1 现场复现 6

3.2 为什么会这样 6

3.3 修复：按「阶段 × 几何来源」联合判定 6

3.4 修复后实测 6

四、D2/D3：SfM 点云污染激光几何（严重） 8

4.1 现象：勾了「稠密重建」反而更差 8

4.2 D3：为什么 novel 永远是 0.0 8

4.3 修复：判定与动作必须一致 8

4.4 修复后实测 9

五、D4/D5：两个「找错地方」的缺陷 10

5.1 X2 色卡定位多了一层 dirname 10

5.2 S1 报错指向项目外的路径 10

六、D6/D7：界面与指标的两个小坑 11

6.1 关闭的可选阶段仍亮灯 11

6.2 coverage_ratio 是被误读的指标 11

七、四组实机实验数据 12

八、修复清单 13

九、校验 14

9.1 测试 14

9.2 关键断言为何这样写 14

十、剩余风险与下一步 15

一、排查结论

此前的排查全部集中在**纯照片场景（场景②）**：照片不被识别、稠密超时、进度页假死、并发互删。本轮把这批问题当作**探针**，逐条投到**带扫描件的场景①（激光几何 + 照片纹理）**上做对照，结论是：**场景① 并不比场景② 安全，它有自己的一组缺陷，而且更隐蔽**——最严重的两个都不会报错，只会让结果悄悄变差。

> **一句话概括**：场景② 的问题多半是「看得见的失败」（卡住、超时、界面不动），而场景① 的问题多是「看不见的劣化」——只有扫描件时整尊判失败（连激光几何都不建模）、勾选稠密后 SfM 点云污染精确几何、色卡定位永远失效、在线标定因为一行漏导入从未生效。四组实机实验与 593 项测试构成本轮证据链。

1.1 七项缺陷一览

编号	缺陷	场景	后果	级别
D1	只有扫描件、没有照片时，S1 抛错导致整尊失败	①	M1~M6 全部不执行，连激光几何都没建模（实测 done=0 failed=1）	致命
D2	有扫描件时勾选「稠密重建」，SfM 点云被无条件并入激光几何	①	M3 三角面 +70%、M5 rmse +34%、纹理覆盖 −60%（实测）	严重
D3	M1 计算 novel 用了 np.asarray，但文件从未 import numpy	①+②	NameError 被静默吞掉，P3 在线标定**从未生效**，novel 恒为 0	严重
D4	X2 色卡定位的照片路径多了一层 dirname	①	永远定位不到色卡，静默退回随机采样，ΔE 长期失真	中
D5	S1 报错只打印回退路径	①	报错指向项目外的 output/00_photo，用户无法据此排查	中
D6	可选阶段关闭时仍打印「□ X1 开始」	①+②	界面亮起永远不会完成的灯，误以为在跑	轻
D7	coverage_ratio 实为「纹理图集像素利用率」	①+②	被误读为「网格纹理覆盖率」，5.87% 看似严重偏低	轻（指标语义）

二、排查方法：把场景② 的问题当探针

场景② 此前定位到的六类问题（识别、超时、并发、进度、续算、乱码），本质上是**编排层与工程层**的问题。这类问题不应假设「换成扫描件就没事」，因此逐条做了映射检验，并按「是否成立 / 是否变形 / 是否引入新问题」分类：

场景② 的问题	在场景① 的表现	判定
照片不被识别（00_photo 不进任务）	allow_photo_only 与 sfm_solve.enabled 联动；场景① 该开关为 true，照片能收进 task	不成立（但若关闭 S1 则 task.photo_paths 为空，仅供 CLI 打印，功能无碍）
稠密超时 3600 s 腰斩	场景① 默认 dense=false 不跑稠密；但界面上可勾选，勾了同样受预算逻辑保护	已由 r6.2 动态预算覆盖
两进程并发互删 dense	同一数据根锁对两场景一视同仁；场景① 若跑稠密同样会互删	已由 r6.2 单实例锁覆盖
进度页假死（30/30 不动）	场景① dense=false 时无深度图进度；S1 稀疏阶段仍有 8 分钟静默	已由 r6.2 心跳 + 子步骤覆盖
超时后不可续算	场景① dense=false 不涉及深度图续算；但稀疏产物仍可能被清	已由 r6.2 位姿指纹覆盖
打包版中文乱码	与场景无关（进程流编码）	已由 r6.2 reconfigure 覆盖
新问题：只有扫描件时无照片	场景② 不存在此情形（照片是唯一输入）；场景① 极常见（只做几何）	**D1：致命，本轮修复**
新问题：双源几何融合	场景② 只有 dense 一个几何源，无需融合；场景① 有两源	**D2/D3：严重，本轮修复**
新问题：多站配准 × 照片位姿	场景② 永远单点云；场景① 常为多站	实测通过（实验 D），但需关注 icp_align 稳定性

映射结果显示：**场景② 的六条问题几乎都被 r6.2 的通用修复覆盖**，但场景① 额外暴露了三个它独有的结构性缺陷（D1/D2/D3），这三个都不是「跑得慢」，而是「跑错了还不说」。

三、D1：只有扫描件时整尊失败（致命）

3.1 现场复现

按向导新建场景① 项目（模板固定 sfm_solve.enabled=true、dense=false、paint_mode=photo），生成示例数据后**删掉 00_photo 目录**，模拟「用户手上只有激光扫描件」这一最常见的情形。实跑结果：

```
▶ S1 开始：demo_relic
失败隔离：item=demo_relic stage=S1 -> RecoverableError（继续下一尊）
  RecoverableError: [E000] S1 照片目录为空或不存在:
  E:\h3d_lablab\output\00_photo\demo_relic
批量处理完成：total=1 done=0 failed=1 skipped=0
```

两个问题同时暴露。第一，**整尊失败**：S1 抛 RecoverableError 后编排层 <code>break</code>，M1~M6 一个都没执行，用户手里明明有精确的激光点云，却连一个几何模型都拿不到。第二，**报错路径是错的**：项目在 <code>output/_scene1_no_photo</code>，报错却指向 <code>output/00_photo</code>——那是 _photo_dir_for 的回退候选，与项目毫无关系，用户照着这个路径去放照片永远不会生效。

3.2 为什么会这样

编排层对「阶段失败」只有一种处理：隔离 + break。这对主线阶段（M1~M6）是对的，但 S1/P1/X1/X2 是**可选增强**——它们在场景① 里只提供相机位姿或附加评估，失败不该废掉激光几何管线。而在场景② 里它们又完全相反：S1 是**唯一几何来源**，失败必须整尊失败，否则 M1 无输入。同一段代码要服务两种语义，原实现只用了一刀切。

3.3 修复：按「阶段 × 几何来源」联合判定

引入 <code>_degrades_on_failure(stage, task)</code>：仅当阶段属于 {S1, P1, X1, X2} **且** 本尊有激光几何（raw_paths 非空）时，失败才降级为「跳过」并继续主线；否则维持原有的失败隔离。

```
OPTIONAL_STAGES = frozenset({"S1", "P1", "X1", "X2"})

def _degrades_on_failure(stage_id, task) -> bool:
    """可选增强阶段失败时，是否降级为「跳过」而不是整尊失败。"""
    return stage_id in OPTIONAL_STAGES and bool(task.raw_paths)
```

降级不是静默：<code>_degrade_optional</code> 会写 WARNING、并在 manifest 里留下 <code>degraded=True</code> 与失败原因，报表可见。同时 M4 也做了配套降级——照片纹理不可用时按《软著方案》2.4「无纹理 → 输出几何模型」退回顶点色，并在 metrics 里留 <code>paint_mode_effective</code> 与 <code>fallback_reason</code>。

3.4 修复后实测

```
▶ S1 开始：demo_relic
[WARNING] 可选阶段降级（主线继续）：item=demo_relic stage=S1
▶ M1 开始：demo_relic
M1 完成：demo_relic（1 站，8000 -> 4363 点）
M3 完成：demo_relic（4363 点 -> 10039 顶点 / 19829 面）
[WARNING] M4 照片纹理不可用，降级为顶点色（几何模型照常交付）
M4 完成：demo_relic（geometry，1 LOD x 2 格式）
M5 完成：demo_relic（coverage=100.0%, rmse=28.17mm -> pass）
M6 完成：demo_relic（2 格式模型 + 元数据快照 + 交付清单）
批量处理完成：total=1 done=1 failed=0 skipped=0
```

从 **done=0 failed=1** 变成 **done=1 failed=0**，几何质量（rmse 28.17 mm）与有照片时（27.15~27.52 mm）基本一致——因为激光几何本来就是主角，照片只负责上色。

四、D2/D3：SfM 点云污染激光几何（严重）

4.1 现象：勾了「稠密重建」反而更差

场景① 的界面上有「稠密重建」复选框。用户直觉是「勾上更完整」，于是 sfm_solve.dense=true，S1 会额外产出 <code>10_dense/<item>/fused_dense.ply</code>，而 M1 里有一段「双源融合」逻辑会把它并进激光几何。同一份示例数据，只改这一个开关，质量指标如下：

指标	仅激光（dense=false）	激光+dense（dense=true）	变化
M1 输入点数	8,000	154,453	×19
M1 输出点数	4,363	6,014	+38%
M3 顶点 / 三角面	10,039 / 19,828	16,936 / 33,758	+69% / +70%
M4 纹理覆盖	5.87%	**2.32%**	**−60%**
M5 几何 rmse	27.52 mm	**36.78 mm**	**+34%**
M5 覆盖率	100%	99.48%	略降

> **关键矛盾**：日志里写着「新信息占比 0.0，权重=reduced」——系统**已经算出** dense 点云零新信息（146453 点全部落在激光点邻域内），却仍然把它们合并了进去。判定说「无用」，动作却是「照合」。这正是 D2 的表层症状。

4.2 D3：为什么 novel 永远是 0.0

追查「新信息占比」的计算，发现问题根子在**一行缺失的 import。**M1 里这样写：

```
if outputs:
    base_pc = load_point_cloud(outputs[0])
    from scipy.spatial import cKDTree
    try:
        d, _ = cKDTree(np.asarray(base_pc.points)).query(
            np.asarray(pcd.points), k=1)        # <-- np 从未导入
        _thr_dist, novel = online_novel_threshold(...)
    except Exception:                           # <-- 把 NameError 吃了
        novel = 0.0
```

<code>runner/__init__.py</code> 全文没有 <code>import numpy as np</code>，所以 <code>np.asarray</code> 抛 NameError；而外层那个<code>except Exception</code> 把它转成 <code>novel = 0.0</code>，连一条日志都不留。后果是：**P3 的在线标定（Otsu 分割 + 指数平滑持久化）从上线起就从未真正执行过**，所有 dense 一律按「novel=0、高度重叠」处理。

验证方式很直接：同一组数据手动调用 <code>online_novel_threshold</code> 得到 **0.4**（远距离 dense）与 **0.0**（完全重合），而 M1 内部对远距离 dense 也报 0.0——说明它根本没算，而是被异常吞掉了。

> **同型缺陷复发**：项目经验里已经记着「禁止静默降级——except 吞异常 + 只留 warning 会掩盖功能实际未生效（P3 增量注册曾因此数周未被发现）」。这次是同一模式的第二次：吞掉的是 NameError，掩盖的是整个在线标定链路。

4.3 修复：判定与动作必须一致

两处改动。第一，补上 <code>import numpy as np</code>，并让 except 记录 <code>dense_novel_error</code> 与 WARNING，不再静默；第二，把「novel 低」的处置从「降权合并」改为**不合并**——既然判定它没有新信息，就不该让它以任何权重进入精确几何。

```
if outputs and novel < _DENSE_MERGE_MIN_NOVEL:     # 0.15
    metrics["dense_merged"] = False
    log.warning("M1 跳过 dense 合并：%s 与激光几何重叠度 %.0f%%"
                "（新信息 %.1f%% < %.0f%% 阈值），SfM 稠密点云无增量信息，"
                "不并入以免污染几何", ...)
else:
    # 场景②（无激光几何）或确有新信息 → 照常合并
    ...
```

4.4 修复后实测

```
M1 跳过 dense 合并：demo_relic 与激光几何重叠度 100%
  （新信息 0.0% < 15% 阈值），SfM 稠密点云无增量信息，不并入以免污染几何
M1 完成：demo_relic（1 站，8000 -> 4363 点）
M3 完成：demo_relic（4363 点 -> 10039 顶点 / 19829 面）
M5 完成：demo_relic（coverage=100.0%, rmse=28.02mm -> pass）
```

指标	修复前（dense=true）	修复后（dense=true）	仅激光基准
M1 输入点数	154,453	8,000	8,000
M3 三角面	33,758	19,829	19,828
M5 rmse	36.78 mm	28.02 mm	27.52 mm
纹理覆盖	2.32%	5.87%	5.87%

修复后带着 dense=true 跑，结果与「只用激光」几乎逐位一致——因为系统正确地识别出这份 dense 没有增量信息。**勾选稠密不再有害**，而真正有增量（例如激光扫不到的凹陷）时仍会正常融合。

五、D4/D5：两个「找错地方」的缺陷

5.1 X2 色卡定位多了一层 dirname

X2 色差评估要在照片里定位 ColorChecker 色卡。它找照片的写法是 <code>os.path.dirname(task.root) + "/00_photo/<item>"</code>，而 <code>task.root</code> **本身就是项目根**（00_raw 的父目录），再套一层 dirname 就跑到项目外面去了。对比其他阶段的写法可以看得很清楚：

位置	照片目录写法	是否正确
S1 sfm_solve._photo_dir_for	task.root/00_photo → 回退 dirname(task.root)/00_photo	正确（优先 task.root）
P1 photo_texture	task_root/00_photo/<item> → 回退上级	正确
M4 photo 模式	调用 P1 同一函数	正确
X2 色差评估	dirname(task.root)/00_photo/<item>	**错误（只有回退路径）**

后果是 X2 **永远定位不到色卡**，静默退回「从纹理图随机采 24 个点」当色卡，ΔE 指标从那一刻起就失真，而且没有任何报错或指标体现——用户在论文里引用的色差数据其实是随机采样的结果。修复为与其他阶段同口径（优先 task.root），并新增 <code>checker_source</code> 指标（photo / texture-random），让降级可见。

5.2 S1 报错指向项目外的路径

S1 在照片目录为空时抛错，但只打印了**解析结果**。而解析逻辑在「00_photo/<item> 不存在」时会回退到上一级目录，于是用户看到的是与项目无关的路径。修复为：报错时同时给出**正确布局**、平铺布局的备选，以及「若只有扫描件请关闭 sfm_solve.enabled」的处置指引：

```
S1 照片目录为空或不存在: <实际解析到的路径>
  正确布局应为：<项目根>\00_photo\<编号>\*.jpg
  （也支持把照片直接放在 <项目根>\00_photo 根下，不建 <编号> 子目录）
  若本尊**只有激光扫描件、没有照片**，请关闭 sfm_solve.enabled；
  此时 S1 会被跳过，M1~M6 照常执行并输出纯几何模型。
```

六、D6/D7：界面与指标的两个小坑

6.1 关闭的可选阶段仍亮灯

编排层在调用阶段函数前就无条件打印「□ X1 开始：<编号>」，而函数第一行就返回 SKIPPED。界面的阶段灯因此会闪一下「运行中」，然后永远不转绿——用户断点续跑时看到 <code>□ P1/X1/X2 开始</code> 却什么都没发生，会怀疑是不是卡住了。**修复**：新增 <code>_stage_enabled</code> 在打印前预判（S1 看 enabled、P1 还要求 paint_mode≠photo、X1/X2 看各自开关），关闭的阶段不再亮灯；主线 M1~M6 恒打印，界面解析当前造像不受影响。

6.2 coverage_ratio 是被误读的指标

该指标实为 <code>coverage.mean()</code>，统计的是 **1024×1024 纹理图集的像素被占用比例**，不是「网格被照片覆盖的比例」。33758 个三角形在 UV 空间只占 5.87% 的像素，完全正常——但报表里写成「纹理覆盖 5.87%」，看起来像 94% 的表面没有纹理。本轮未改动计算（避免动已验证的纹理链路），但已在文档与结论中明确口径，并建议下一轮拆成 <code>atlas_utilization</code>（图集利用率）与 <code>mesh_coverage</code>（网格着色比例）两个指标。

七、四组实机实验数据

全部使用内置示例生成器（合成椭球 8,000 点激光 + 16 张 640px 合成照片），RTX 3050 Laptop，COLMAP 4.1.1。四组实验覆盖场景① 的四条主路径：

实验	场景	关键配置	结果	状态
A	① 标准（激光 + 照片）	dense=false, paint_mode=photo	S1 56.0 s（16/16 注册）→ M1 4,363 点 → M3 19,829 面 → M4 照片投影（16 相机）→ M5 27.15 mm pass → M6 完成	done=1
B	① 只有扫描件（删 00_photo）	同上	修复前：S1 抛错 → done=0 failed=1； 修复后：S1 降级 → M1~M6 全通，M4 降级顶点色，M5 28.17 mm pass	done=1（修复后）
C	① + 勾选稠密	dense=true	修复前：并入 146,453 点 → 33,758 面 / 36.78 mm / 覆盖 2.32%； 修复后：跳过合并 → 19,829 面 / 28.02 mm / 覆盖 5.87%	done=1
D	① 多站（2 站扫描件）	dense=false	S1 62.8 s → M1 2 站 14,084→7,665 点 → M2 配准 fitness=1.000 → M3 24,863 面 → M4 照片投影 → M5 22.00 mm pass	done=1

实验 D 是场景② 完全不存在、而场景① 必跑的路径：多站激光配准（M2）与照片位姿（S1）叠加。实测 M2 把第二站配准到首站坐标系后，M4 的 <code>icp_align</code> 仍能把 SfM 位姿对齐到几何坐标系，照片投影正常、rmse 22.00 mm 优于单站——说明配准后的坐标系变化没有破坏纹理链路。

八、修复清单

文件	改动	对应缺陷
runner/batch_processor.py	新增 OPTIONAL_STAGES / _degrades_on_failure / _degrade_optional / _degraded_entry；两个异常分支与多进程路径同步按场景分流；新增 _stage_enabled 预判「□开始」日志	D1、D6
runner/__init__.py	补 import numpy；M1 新增 dense 合并门控（novel<0.15 且有激光 → 不合并）；M4 照片纹理失败降级顶点色并留 metrics；X2 照片目录改优先 task.root 并新增 checker_source	D2、D3、D4
sfm_solve.py	S1 照片目录为空时，报错给出正确布局 / 平铺备选 / 关闭开关指引	D5

九、校验

9.1 测试

新增 <code>tests/test_runner/test_scene1_degrade.py</code>，共 25 项，覆盖七个方面：降级判定（谁能降级/谁必须失败）、场景① 无照片的调度行为、降级标记落 manifest、非 Recoverable 异常同样降级、M1 双源融合门控（真实点云：重合不合并 / 远离照常合并 / 纯照片必须合并）、S1 报错文案、X2 照片目录定位与 checker_source、可选阶段亮灯预判。

测试组	项数	锁定行为
TestDegradeDecision	9	S1 有激光可降级、纯照片必须失败、主线恒不降级
TestScene1WithoutPhotos	4	无照片整尊完成 / 纯照片仍失败 / manifest 留 degraded
TestDenseMergeGate	3	重合不合并、远离照常合并、纯照片必须合并
TestStageStartLog	4	关闭的可选阶段不亮灯、S1 开启仍打「□ S1 开始」
TestS1ErrorMessage	1	报错含正确布局与关闭开关指引
TestX2PhotoDir	1	X2 能遍历到 <root>/00_photo/<item> 下的照片

全量回归：593 passed / 0 failed（基线 568 + 本轮 25）。四组端到端实验全部 done=1。

9.2 关键断言为何这样写

测试不满足于「不报错」，而是锁**行为本质**：例如 <code>test_overlapping_dense_is_not_merged</code> 断言 <code>points_in == len(base)</code>（dense 点确实没进统计）而不是只断言 <code>dense_merged is False</code>；<code>test_photo_only_item_always_merges_dense</code> 断言产物文件真实存在；<code>test_x2_locates_photos_under_task_root</code> 用打桩记录 <code>locate_colorchecker</code> 的调用路径——若目录找错，桩根本不会被调用。

十、剩余风险与下一步

优先级	事项	说明
P1	X2 的 ΔE 需要真实色卡照片复核	本轮只修了路径与可见性；具体数值仍依赖真实 ColorChecker 拍摄
P1	场景① 真实扫描件（多站、非合成）实机校验	本轮用合成椭球，M2 fitness=1.000 属理想值；真实点云需关注配准残差
P1	dense 真有增量时的融合效果未验证	门控只在「零新信息」侧做了实测；「有增量」侧仅有单元测试
P2	coverage_ratio 拆分为 atlas_utilization + mesh_coverage	当前口径易被误读；拆分需同步改报表与论文表格
P2	icp_align 的 scale_s 达 1.15 说明 SfM 尺度与激光差异明显	当前靠 ICP 自动估尺度兜底；若控制点充足应优先用控制点定标
P3	任务发现无「重复照片指纹」检测	场景① 下同一批照片被多尊重复跑 S1（稀疏约 1 分钟/尊，规模大时显著）

本轮交付边界：D1~D6 已修复并实测，D7 已在文档中澄清口径但未改计算。场景① 的「标准 / 无照片 / 勾稠密 / 多站」四条主路径全部跑通，全量 593 项测试 0 失败。建议下一步用真实扫描件（2 站以上）复跑一次实验 D，重点看 M2 配准残差与 M4 照片投影的稳定性。